Hands-on OOP Project: Interfaces

Building a Hybrid Block Encryption Algorithm (HBEA) in Java

Item	Details
Work mode	Individual
Prerequisites	Classes and objects, encapsulation, inheritance, polymorphism, abstract classes, interfaces, arrays
Deliverables	Project archive (.zip), individual .java files, short report with screenshots (see Section 8)
Due date	Monday 26 Oct
Submission	Blackboard (designated assignment link)

1. Overview

You are part of a team building a small **secure messaging tool**. A user types a short message, the tool encrypts it with a key derived from the user's e-mail address, and only someone holding the same key can decrypt it.

The team wants the messaging part of the application (console today, a WebApp GUI later) to work with **any** encryption algorithm, so that one algorithm can be replaced by another without touching the rest of the code. In Java, that is exactly what **interfaces** are for. Your job is to:

1. Design a small set of **interfaces** that describe what a cipher, a key generator and a round function must do.
2. Implement the **Hybrid Block Encryption Algorithm (HBEA)**: a block cipher built from SHA-256 and the classic DES tables, behind those interfaces.
3. Prove the design works by plugging a **second, different cipher** into the same application without changing it.

2. Learning Objectives

By completing this project you will be able to:

- Define interfaces with abstract methods, a **default** method and constants, and implement them in concrete classes.
- Use an **abstract class** to share common logic (a template) while leaving key steps to subclasses.
- Apply **inheritance, method overriding and overloading** in a meaningful design.
- Write code that depends on an **interface type**, not on a concrete class ("program to an interface").
- Protect sensitive data (the key) through **encapsulation**.
- Validate input and report problems with **exceptions**, and test your code using **TDD** and **BDD** style tests.
- Work with bits and bytes in Java (byte, & 0xFF, shifts, XOR).

3. Background

3.1 What is encryption?

Encryption converts readable data (**plaintext**) into unreadable data (**ciphertext**) using a secret **key**. Decryption reverses the process with the same key. People have tried to keep messages secret for centuries; a famous mechanical example is the Enigma machine.

3.2 Block ciphers: from DES to AES

A **block cipher** encrypts fixed-size blocks of bits. The Data Encryption Standard (DES) encrypted 64-bit blocks and was the standard for more than two decades. It was replaced by the Advanced Encryption Standard (AES), which protects Wi-Fi, HTTPS, encrypted USB drives and many other systems today.

3.3 The Feistel idea (why decryption is possible)

DES uses a **Feistel network**. Each 64-bit block is split into a left half **L** and a right half **R**. In each round, only the left half is changed, by XOR-ing it with the output of a round function **F** applied to the right half:

```
L(i) = R(i-1)

R(i) = L(i-1) XOR F(R(i-1), RK(i))
```

Because x XOR y XOR y = x, the same structure run with the round keys in **reverse order** undoes the encryption. This works **even though F itself cannot be reversed** (the S-boxes turn 6 bits into 4 bits, so information is lost inside F). HBEA uses exactly this idea.

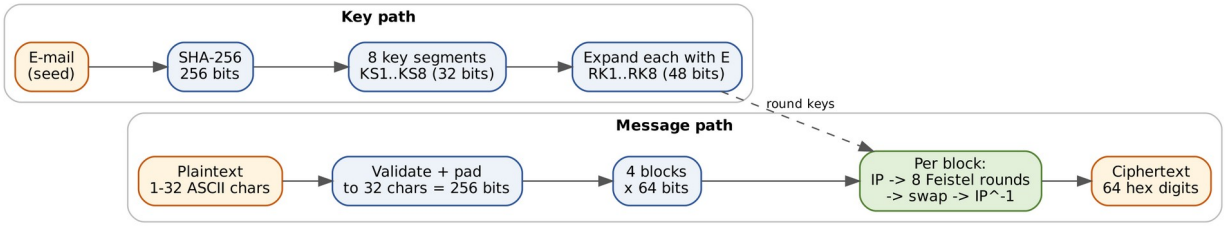


Figure 1. HBEA at a glance: the key path and the message path.

4. HBEA Algorithm Specification

> **Read this section carefully**
>
> Your output must match the test vectors in Section 7 **exactly**. That is only possible if you follow the conventions below.

4.1 Conventions

- **Bits** are stored as int values 0 or 1 in an int[] (or as a String of "0"/"1"). Bit 1 is the **leftmost (most significant)** bit.
- **Bytes to bits:** each byte becomes 8 bits, most significant bit first; bytes are processed in order.
- **Tables** (Appendix A) use 1-based positions. For a table T, output bit i = input bit T[i]. In Java (0-based arrays): out[i] = in[T[i] - 1].
- **Ciphertext** is shown as **64 uppercase hexadecimal digits** (256 bits).

4.2 Step 1: Key generation

1. Take the seed: your **official e-mail address**, trimmed and converted to lower case.
2. Compute **SHA-256** of its UTF-8 bytes. The result is a 256-bit (32-byte) key.
3. Split the key into **8 key segments** KS1 ... KS8 of **32 bits** each (KS1 = the first 32 bits).
4. Expand each key segment to a **48-bit round key** using the Expansion table E: RK(i) = E(KS(i)).
5. Keep the key material **private** inside the cipher object. It must never be returned by a public getter or printed (except in test code).

Item	Size	Notes
SHA-256 key	256 bits	32 bytes = 64 hex digits

Item	Size	Notes
Key segments KS1 … KS8	8 × 32 bits	Consecutive slices of the key
Round keys RK1 … RK8	8 × 48 bits	RK(i) = E(KS(i)); one per round

4.3 Step 2: Message processing

1. **Validate** the message: it must not be empty, must contain only printable ASCII characters (codes 32 to 126), and must be at most 32 characters. Otherwise throw an InvalidMessageException (a checked exception you write).
2. **Pad** a message shorter than 32 characters with the NUL character '\0' (code 0) until it has exactly 32 characters = **256 bits**. NUL never appears in valid input, so it can be removed safely after decryption.
3. Split the 256 bits into **8 message segments** M-Seg1 … M-Seg8 of 32 bits. Each **pair** of segments forms one **64-bit block**: Block 1 = M-Seg1 + M-Seg2, …, Block 4 = M-Seg7 + M-Seg8.

Item	Size	Notes
Padded message	256 bits	32 ASCII characters
Message segments M-Seg1 … M-Seg8	8 × 32 bits	
Blocks 1 … 4	4 × 64 bits	Each block is encrypted independently

4.4 Step 3: Encrypting one 64-bit block

1. Apply the **Initial Permutation**: X = IP(block).
2. Split X into L0 (bits 1–32) and R0 (bits 33–64).
3. Run **8 rounds** (i = 1 … 8): L(i) = R(i-1) and R(i) = L(i-1) XOR F(R(i-1), RK(i)).
4. After round 8, **swap** the halves: preOutput = R8 + L8 (R8 first).
5. Apply the **Final Permutation** (inverse of IP): cipherBlock = IP^{-1}(preOutput).

The round function F works on the 32-bit right half (Figure 2):

```
F(R, RK) = P( S( E(R) XOR RK ) )

E(R)      : expand 32 -> 48 bits with the Expansion table
XOR RK    : XOR with the 48-bit round key
S(...)    : 8 S-boxes, 48 -> 32 bits
P(...)    : permute the 32 bits with the Permutation table P
```

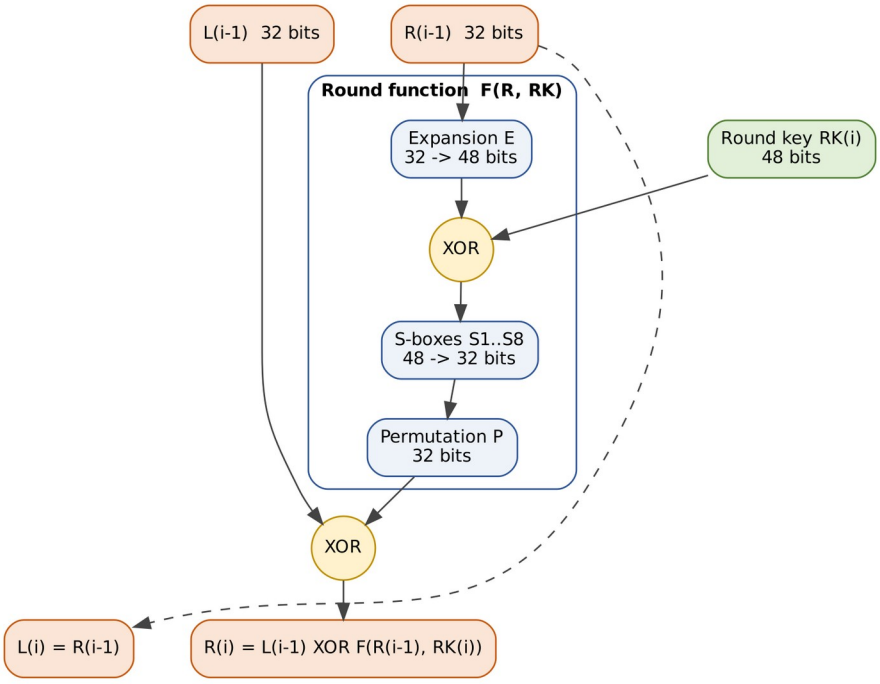


Figure 2. One HBEA round (repeated 8 times per block).

How an S-box lookup works

Split the 48 bits into 8 groups of 6 bits; group j uses S-box j. For a group b1 b2 b3 b4 b5 b6:

- **row** = the 2-bit number b1 b6 (0–3)
- **column** = the 4-bit number b2 b3 b4 b5 (0–15)
- the table value (0–15) is written as **4 bits**, most significant first.

Example (from the test vector): the first 6 bits are 001101. Row = 0 1 = 1, column = 0110 = 6, and S1[1][6] = 13 = 1101.

4.5 Step 4: Decryption

Decryption uses the **same block procedure** (IP, 8 rounds, swap, IP^{-1}) on each ciphertext block, but with the round keys in **reverse order**: RK8 in round 1, RK7 in round 2, ..., RK1 in round 8. Finally, convert the 256 bits back to characters and remove the trailing NUL padding.

> **Common mistake**
>
> Do **not** try to "invert" the S-boxes. They cannot be inverted. The Feistel structure (Section 3.3) is what makes decryption work.

5. Required Object-Oriented Design

This is the core of the assignment. Your code must be organised around the three interfaces below. Use the exact names and method signatures so that the instructor's test program can run against your classes. You may add private helper methods and extra classes.

5.1 Interfaces

```
public interface Cipher {
    String encrypt(String plaintext) throws InvalidMessageException; // 64 hex digits
    String decrypt(String cipherHex);
    String getName();

    // default method: every cipher gets a self-check for free
    default boolean verifyRoundTrip(String plaintext) throws InvalidMessageException {
```

```
        return plaintext.equals(decrypt(encrypt(plaintext)));
    }
}

public interface KeyGenerator {
    int SEGMENTS = 8;         // interface constants are public static final
    int SEGMENT_BITS = 32;
    int[][] generateKeySegments(String seed); // 8 x 32 bits
}

public interface RoundFunction {
    int[] apply(int[] right32, int[] roundKey48); // returns 32 bits
}
```

5.2 Classes

Class	Responsibility	OOP concept shown
AbstractBlockCipher (abstract, implements Cipher)	Validation, padding, splitting into 64-bit blocks, hex conversion. Declares encryptBlock(int[]) and decryptBlock(int[]) as **abstract**.	Abstraction, template method, implementing an interface
HbeaCipher (extends AbstractBlockCipher)	Builds the 8 round keys and runs IP → 8 Feistel rounds → swap → IP^{-1}. Receives a KeyGenerator and a RoundFunction in its constructor.	Inheritance, overriding, composition through interfaces, encapsulation of the key
XorCipher (extends AbstractBlockCipher)	A deliberately simple second cipher: XOR each block with the first 64 key bits.	Polymorphism: a second implementation of the same contract
Sha256KeyGenerator (implements KeyGenerator)	Step 1 of the algorithm (Section 4.2), up to the 8 key segments.	Implementing an interface
DesRoundFunction (implements RoundFunction)	F(R, RK) = P(S(E(R) XOR RK)).	Implementing an interface
BitUtils (final, static methods)	permute, xor, bytesToBits, bitsToBytes, toHex, fromHex. xor throws IllegalArgumentException if lengths differ.	Utility class, reuse
HbeaTables (final)	The DES tables from Appendix A as private static final arrays.	Constants, encapsulation
InvalidMessageException	Checked exception for invalid input.	Custom exceptions
SecureMessenger	The "application": has a Cipher field and send / receive methods. Knows **nothing** about HBEA.	Programming to an interface
Main	Console demo: reads e-mail and message, encrypts, decrypts, prints results for **both** ciphers.	

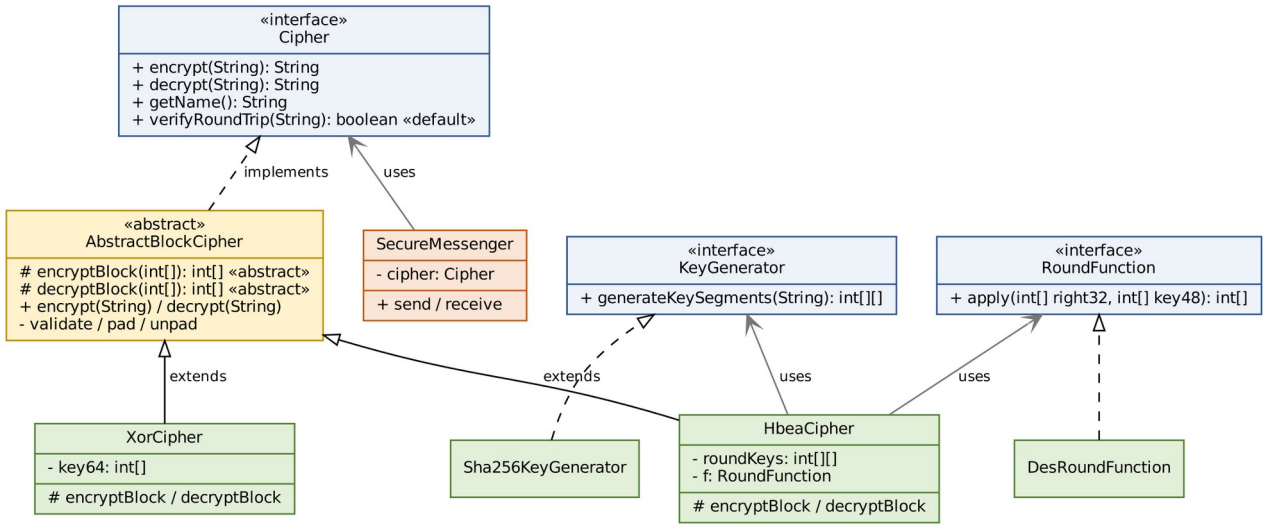


Figure 3. Suggested class diagram (dashed arrow = implements, solid hollow arrow = extends).

5.3 Design rules

- SecureMessenger and Main must refer to ciphers through the **`Cipher`** type only, e.g. Cipher c = new HbeaCipher(email);.
- Swapping HbeaCipher for XorCipher must require **changing one line** (the new ... expression) and nothing else.
- Provide at least one **overloaded** method, e.g. encrypt(String) and encrypt(int[] bits256) in AbstractBlockCipher.
- Use @Override on every overriding method.
- Key material and round keys are private final fields. No public getter returns them.

6. Implementation Tasks

Build and test the project in this order. Each task maps to a rubric criterion (Section 9).

Task 1: Input validation and padding (TDD)

Write the tests first, then the code. Test: empty input, null, more than 32 characters, a non-ASCII character such as "é", a short message (padding applied), exactly 32 characters (no padding).

Task 2: Key generation (BDD)

Describe the behaviour as Given–When–Then scenarios, for example:

Given the seed "student@student.ie.edu", When the key is generated, Then KS1 is 6C099BA0

Given " Student@Student.IE.edu " (spaces, capitals), Then the same key is produced

Task 3: XOR (TDD + exceptions)

Test BitUtils.xor with known values, with arrays of different lengths (must throw IllegalArgumentException), and with null. Callers must handle the exception gracefully.

Task 4: Round function and one round (unit tests)

Unit-test E, the S-box lookup, P and F separately against Section 7. Then, as in the class demo, encrypt a block that starts with the **first letter of your name** and print L and R after each round.

Task 5: Full encryption and decryption (BDD)

Given a key and a message, When I encrypt and then decrypt, Then I get the original message back. Also show that decrypting with a **different** e-mail does **not** return the message.

Task 6: Polymorphism demo

Run SecureMessenger with HbeaCipher and then with XorCipher, without changing SecureMessenger. Print getName() and the round-trip result for each.

> **Testing tools**
> You may use JUnit 5 if your IDE supports it, or write simple test methods in a Tests class that print PASS/FAIL. Either is acceptable, as long as tests are written **before** (TDD) or **as behaviour scenarios** (BDD) and their results are shown in your report.

7. Test Vectors (check your work)

Seed: student@student.ie.edu Message: Meet me at the library at 5 PM!! (32 characters). All values in hexadecimal.

Value	Expected result
SHA-256 key	6C099BA09261A2A3516842CB93139F39 1751408747F298EFD359D5E855D4B9F0
KS1 … KS4	6C099BA0 9261A2A3 516842CB 93139F39
KS5 … KS8	17514087 47F298EF D359D5E8 55D4B9F0
RK1 = E(KS1)	358053CF7D00
Block 1 ("Meet me ")	4D656574206D6520
IP(Block 1)	L0 = 6F086F67 R0 = 00FE2100
E(R0)	0017FC102800
E(R0) XOR RK1	3597AFDF5500
S-box output	D688919D
F(R0, RK1) after P	6B80A973
After round 1	L1 = 00FE2100 R1 = 0488C614
Ciphertext	AB980FB8275358AE1D411D5FF38B89E5 DCAD224E7282B8A8FF7A945EE97A5D40
Decrypted text	Meet me at the library at 5 PM!!

8. Deliverables and Submission

- **Project archive (.zip)** containing the complete source code (all packages) that compiles and runs.
- **Individual .java files** uploaded separately, each with a Javadoc comment at the top and the **results of running it** pasted as a comment at the end of the file.
- **Short report (PDF, 2–4 pages)**: your class diagram, a short explanation of each step (key generation, message processing, encryption, decryption), how you used interfaces and the abstract class, your test results, and your answers to the reflection questions below.
- **Appendix C screenshots** showing: the input (e-mail and message), the ciphertext, the decrypted text, and the polymorphism demo with both ciphers.
- **Use your own official e-mail address** as the seed in your screenshots.

Reflection questions (answer briefly in the report)

1. The S-boxes cannot be reversed. Explain in your own words why decryption still works.
2. Encrypt the short message "Hi Anas". Why are the last three ciphertext blocks identical? What does this reveal about encrypting blocks independently?

3. Why is a key derived only from an e-mail address weak in real life?
4. Suppose the team wants to switch to AES using Java's javax.crypto library. Which classes would you add or change, and which would stay untouched? What does this show about interfaces?

9. Grading Rubric

Criterion	Excellent (A)	Good (B)	Satisfactory (C)	Needs Improvement (D/F)	Weight
1. Input handling (TDD)	Complete validation with proper exceptions and padding; TDD tests cover all edge cases (length < 32, > 32, empty, invalid characters).	Validation works but misses one edge case or TDD evidence is unclear.	Partial validation; exceptions or padding incomplete.	No validation or testing.	15%
2. Key generation (BDD)	Follows Section 4.2; Given–When–Then scenarios; output matches the test vectors and is reproducible.	BDD tests present but one scenario missing or unclear.	Works but not verified with BDD.	Incorrect or missing.	15%
3. Encryption round (unit tests)	Round matches Figure 2 and Section 7; well-structured unit tests; first-letter-of-name demo works.	Logic mostly correct; one test or the demo missing.	Partial round; limited testing.	Missing or incorrect.	20%
4. XOR (TDD + exceptions)	Thoroughly tested; length mismatch and null handled with exceptions.	Correct but limited tests.	Partly works; little TDD evidence.	Incorrect or untested.	15%
5. Decryption (BDD)	Correctly reverses encryption; BDD scenarios incl. wrong-key case.	Works but tests poorly documented.	Incomplete or partly verified.	Not implemented or incorrect.	15%
6. OOP design: interfaces	All three interfaces implemented as specified, incl. default method; abstract class with abstract methods; inheritance, overriding and overloading; SecureMessenger uses only Cipher; XorCipher swap works; key encapsulated.	Design mostly correct; one element missing (e.g., second cipher or overloading).	Some OOP elements, but code depends on concrete classes.	Procedural design; no interfaces.	15%
7. Code quality and documentation	Clean, readable, fully commented; report explains logic and tests clearly; reflection questions answered.	Readable; minimal comments.	Runs but poorly explained.	Hard to read or undocumented.	5%

> **Security note**
>
> HBEA is a **learning exercise**, not a secure algorithm: 8 rounds, a key derived from a public value, and independent block encryption are all weaknesses. In real software, always use well-tested libraries (for example AES-GCM via javax.crypto) and never invent your own cipher.

Appendix A: DES Tables (copy into HbeaTables.java)

All positions are 1-based (see Section 4.1).

Initial Permutation IP (64 → 64)

```
static final int[] IP = {
   58, 50, 42, 34, 26, 18, 10, 2,   60, 52, 44, 36, 28, 20, 12, 4,
   62, 54, 46, 38, 30, 22, 14, 6,   64, 56, 48, 40, 32, 24, 16, 8,
   57, 49, 41, 33, 25, 17,  9, 1,   59, 51, 43, 35, 27, 19, 11, 3,
   61, 53, 45, 37, 29, 21, 13, 5,   63, 55, 47, 39, 31, 23, 15, 7 };
```

Final Permutation IP^{-1} (64 → 64)

```
static final int[] FP = {
   40, 8, 48, 16, 56, 24, 64, 32,   39, 7, 47, 15, 55, 23, 63, 31,
   38, 6, 46, 14, 54, 22, 62, 30,   37, 5, 45, 13, 53, 21, 61, 29,
   36, 4, 44, 12, 52, 20, 60, 28,   35, 3, 43, 11, 51, 19, 59, 27,
   34, 2, 42, 10, 50, 18, 58, 26,   33, 1, 41,  9, 49, 17, 57, 25 };
```

Expansion E (32 → 48)

```
static final int[] E = {
   32,  1,  2,  3,  4,  5,    4,  5,  6,  7,  8,  9,
    8,  9, 10, 11, 12, 13,   12, 13, 14, 15, 16, 17,
   16, 17, 18, 19, 20, 21,   20, 21, 22, 23, 24, 25,
   24, 25, 26, 27, 28, 29,   28, 29, 30, 31, 32,  1 };
```

Permutation P (32 → 32)

```
static final int[] P = {
   16,  7, 20, 21,  29, 12, 28, 17,   1, 15, 23, 26,   5, 18, 31, 10,
    2,  8, 24, 14,  32, 27,  3,  9,  19, 13, 30,  6,  22, 11,  4, 25 };
```

S-boxes S1 ... S8 (S[box][row][column])

```
static final int[][][] S = {
 { {14, 4,13, 1, 2,15,11, 8, 3,10, 6,12, 5, 9, 0, 7},     // S1
   { 0,15, 7, 4,14, 2,13, 1,10, 6,12,11, 9, 5, 3, 8},
   { 4, 1,14, 8,13, 6, 2,11,15,12, 9, 7, 3,10, 5, 0},
   {15,12, 8, 2, 4, 9, 1, 7, 5,11, 3,14,10, 0, 6,13} },
 { {15, 1, 8,14, 6,11, 3, 4, 9, 7, 2,13,12, 0, 5,10},     // S2
   { 3,13, 4, 7,15, 2, 8,14,12, 0, 1,10, 6, 9,11, 5},
   { 0,14, 7,11,10, 4,13, 1, 5, 8,12, 6, 9, 3, 2,15},
   {13, 8,10, 1, 3,15, 4, 2,11, 6, 7,12, 0, 5,14, 9} },
 { {10, 0, 9,14, 6, 3,15, 5, 1,13,12, 7,11, 4, 2, 8},     // S3
   {13, 7, 0, 9, 3, 4, 6,10, 2, 8, 5,14,12,11,15, 1},
   {13, 6, 4, 9, 8,15, 3, 0,11, 1, 2,12, 5,10,14, 7},
   { 1,10,13, 0, 6, 9, 8, 7, 4,15,14, 3,11, 5, 2,12} },
 { { 7,13,14, 3, 0, 6, 9,10, 1, 2, 8, 5,11,12, 4,15},     // S4
   {13, 8,11, 5, 6,15, 0, 3, 4, 7, 2,12, 1,10,14, 9},
   {10, 6, 9, 0,12,11, 7,13,15, 1, 3,14, 5, 2, 8, 4},
   { 3,15, 0, 6,10, 1,13, 8, 9, 4, 5,11,12, 7, 2,14} },
 { { 2,12, 4, 1, 7,10,11, 6, 8, 5, 3,15,13, 0,14, 9},     // S5
   {14,11, 2,12, 4, 7,13, 1, 5, 0,15,10, 3, 9, 8, 6},
   { 4, 2, 1,11,10,13, 7, 8,15, 9,12, 5, 6, 3, 0,14},
   {11, 8,12, 7, 1,14, 2,13, 6,15, 0, 9,10, 4, 5, 3} },
 { {12, 1,10,15, 9, 2, 6, 8, 0,13, 3, 4,14, 7, 5,11},     // S6
   {10,15, 4, 2, 7,12, 9, 5, 6, 1,13,14, 0,11, 3, 8},
   { 9,14,15, 5, 2, 8,12, 3, 7, 0, 4,10, 1,13,11, 6},
   { 4, 3, 2,12, 9, 5,15,10,11,14, 1, 7, 6, 0, 8,13} },
 { { 4,11, 2,14,15, 0, 8,13, 3,12, 9, 7, 5,10, 6, 1},     // S7
   {13, 0,11, 7, 4, 9, 1,10,14, 3, 5,12, 2,15, 8, 6},
   { 1, 4,11,13,12, 3, 7,14,10,15, 6, 8, 0, 5, 9, 2},
```

```
    { 6,11,13, 8, 1, 4,10, 7, 9, 5, 0,15,14, 2, 3,12} },
  { {13, 2, 8, 4, 6,15,11, 1,10, 9, 3,14, 5, 0,12, 7},    // S8
    { 1,15,13, 8,10, 3, 7, 4,12, 5, 6,11, 0,14, 9, 2},
    { 7,11, 4, 1, 9,12,14, 2, 0, 6,10,13,15, 3, 5, 8},
    { 2, 1,14, 7, 4,10, 8,13,15,12, 9, 0, 3, 5, 6,11} } };
```

Appendix B: Java Toolbox

Goal	Java API	Notes
SHA-256 hash	MessageDigest.getInstance("SHA-256") md.digest(byte[] input)	Package java.security. getInstance throws the checked NoSuchAlgorithmException.
String → bytes	text.getBytes(StandardCharsets.US_ASCII) seed.getBytes(StandardCharsets.UTF_8)	Package java.nio.charset. Always name the charset.
Byte → unsigned int	int v = b & 0xFF;	Java bytes are signed (−128...127).
Read bit k of v (k = 7 is leftmost)	(v >> k) & 1	
Int → 8-bit binary string	String.format("%8s", Integer.toBinaryString(v)).replace(' ', '0')	toBinaryString drops leading zeros, so pad it.
XOR of two bits	a ^ b	
Bytes → hex	String.format("%02X", b & 0xFF)	Use a StringBuilder in a loop.
Hex → int	Integer.parseInt("6C", 16)	
Copy part of an array	Arrays.copyOfRange(bits, from, to)	Package java.util.

Docs: MessageDigest (Java SE 21) · DES supplementary material (tables)

```
// Example: SHA-256 of the seed
byte[] hash;
try {
   MessageDigest md = MessageDigest.getInstance("SHA-256");
   hash = md.digest(seed.trim().toLowerCase().getBytes(StandardCharsets.UTF_8));
} catch (NoSuchAlgorithmException e) {
   throw new IllegalStateException("SHA-256 is not available", e);
}
```

Appendix C: Execution Screenshots

Paste your screenshots here (or in your report), each with a one-line caption:

- Input: your e-mail address and your message.
- Output: the ciphertext (64 hex digits) and the decrypted message.
- Wrong-key test: decrypting with a different e-mail.
- Polymorphism demo: SecureMessenger running with HbeaCipher and with XorCipher.
- Test results: TDD/BDD tests passing.